

PUC MINAS · GCES · 2026

Biblioteca API

Gerenciamento de Configuração de Software
Processos · Ferramentas · Pipeline Jenkins

👑 Julia Medeiros

💻 Gustavo Delfino

💻 Matheus Caetano

💻 Leticia Figueiredo

💻 Thiago Laass



O Projeto & Contexto



O QUE É

Biblioteca API

API REST em **Python 3.12 + FastAPI** para gerenciamento de acervo, usuários e empréstimos de uma biblioteca digital.



OBJETIVO DO PLANO DE GC

Garantir que o software seja desenvolvido, versionado, testado e implantado de forma **controlada, rastreável e repetível**, alinhado aos princípios de CI/CD e DevOps.

5

Integrantes

4

Stages CI/CD

3

Níveis de Teste



ESCOPO DO PLANO DE GCS

- ✦ Código-fonte da aplicação FastAPI
- ✦ Artefatos de build e testes (Makefile)
- ✦ Pipeline CI/CD com **Jenkins**
- ✦ Empacotamento e deploy via **Docker**
- ✦ Controle de mudanças via branch `main`
- ✦ Webhook GitHub → Jenkins via **ngrok**



STACK PRINCIPAL

Git + GitHub

Jenkins

Docker

pytest

FastAPI · Uvicorn · Pydantic v2

Make · ngrok · Python 3.12

Itens de Configuração (ICs)

IC	LOCALIZAÇÃO	DESCRIÇÃO
Código-fonte	app/	Models, services, repositories, API REST (endpoints)
Testes	tests/unit/ tests/integration/ tests/acceptance/	Suítes pytest automatizadas em 3 camadas
Dependências	requirements.txt	Bibliotecas Python com versões exatas fixadas
Build	Makefile · Dockerfile	Automação de compilação e empacotamento multi-stage
Pipeline	Jenkinsfile	Definição declarativa do pipeline Jenkins
Deploy	scripts/deploy.sh	Script de implantação automatizada em Docker
Infra CI	docker-compose.jenkins.yml scripts/setup-*.sh	Setup do Jenkins e ngrok
Docs	README.md	Instruções de execução, endpoints, setup do pipeline



MÉTODO DE IDENTIFICAÇÃO

- ✦ Cada IC identificado por **caminho no repositório**
- ✦ Versão rastreada pelo **hash do commit Git**
- ✦ Dependências pinadas em `requirements.txt` — builds **reproduzíveis**
- ✦ Artefato Docker tagueado com `BUILD_NUMBER` do Jenkins

Linhas de Base (Baselines)



O QUE É UMA BASELINE?

Conjunto de versões de ICs **comprovadamente estáveis**. Usada como base para a próxima fase do desenvolvimento, com mudanças controladas formalmente.

BASELINES DO PROJETO

BASELINE	MOMENTO	CONTEÚDO
Inicial	Initial commit	Estrutura base, modelos, serviços e testes
CI	Commit do Jenkinsfile	Pipeline completo Build → Deploy
Release	Jenkins Build #6+	Container Docker rodando + health check OK
Entrega	28/06/2026	Plano PDF + repositório ZIP + slides

MARCOS DO PROJETO

- M1 — API Base** ✓
CRUD de livros, usuários e empréstimos + testes passando
- M2 — Pipeline CI** ✓
Jenkins: Build, Test, Acceptance com SUCCESS
- M3 — Deploy Automatizado** ✓
Deploy em Docker via pipeline · health check OK
- M4 — Webhook GitHub** ✓
Push na `main` dispara build sem intervenção manual
- M5 — Entrega Final** **hoje**
PDF + ZIP + deploy script + apresentação

Controle de Mudanças & CCB

FLUXO DE CONTROLE DE MUDANÇAS

01 **Solicitação de Mudança**
Desenvolvedor propõe alteração via issue ou discussão no grupo



02 **Análise de Impacto (CCB)**
Equipe verifica se a mudança afeta pipeline, API, testes ou deploy



03 **Implementação & Teste Local**
Código alterado e validado localmente antes do push



04 **Commit & Push na main**
Webhook dispara o pipeline Jenkins automaticamente



05 **Validação Automática**
SUCCESS = aprovado automaticamente **FAILURE** = hotfix obrigatório



CCB — CONSELHO DE CONTROLE DE MUDANÇAS

Todos os 5 integrantes

- ✦ Reuniões informais (presencial ou online)
- ✦ Aprovação por **consenso da equipe**
- ✦ Decisões registradas nos **commits Git** ou README
- ✦ Build quebrado na **main** = hotfix **prioritário** antes de novas features



RELATÓRIOS & AUDITORIAS

RELATÓRIO	FONTE	FREQUÊNCIA
Status pipeline	Jenkins UI	A cada push
Cobertura de testes	pytest-cov	A cada build
Webhook log	GitHub Deliveries	A cada push
Log de deploy	Console Jenkins	A cada release
Health check	GET /api/v1/health	Após deploy

Organização, Papéis & Infraestrutura

ESTRUTURA DE PAPÉIS DE CM

PAPEL	QUEM	RESPONSABILIDADE PRINCIPAL
Gerente de Config.	Julia	Mantém o plano, configura Jenkins e repositório Git
Desenvolvedores	Gustavo · Matheus · Leticia · Thiago	Implementam código, escrevem testes, fazem commits revisados
DevOps / Integrador	Gustavo · Matheus · Leticia · Thiago	Mantêm Jenkinsfile, Docker e scripts de deploy
CCB	Todos	Aprovam mudanças significativas antes do merge na <code>main</code>



AMBIENTE & INFRAESTRUTURA

- ♦ Equipe distribuída — máquinas **Linux locais**
- ♦ Sem servidor cloud: Jenkins roda **localmente em Docker**
- ♦ Acesso externo via **ngrok** (túnel HTTPS gratuito)
- ♦ **GitHub** como repositório central de armazenamento
- ♦ Repositório < 50 MB · Imagens Docker ~300 MB

FERRAMENTAS DE SCM

- ♦ **Git** — controle de versão local + histórico completo
- ♦ **GitHub** — hospedagem remota + webhooks + colaboração
- ♦ **Jenkins** — integração e entrega contínua
- ♦ **Docker** — empacotamento e isolamento de ambiente

Jenkins — Visão Geral & Arquitetura



O QUE É O JENKINS NO PROJETO

- ✦ Servidor de **Integração e Entrega Contínua (CI/CD)**
- ✦ Executa builds automáticos a cada `git push` na `main`
- ✦ Pipeline configurado como **código** no `Jenkinsfile`
- ✦ Mantém histórico de builds, logs e artefatos
- ✦ Roda em container Docker via `docker-compose.jenkins.yml`



INTEGRAÇÃO GITHUB → JENKINS

- ✦ GitHub dispara webhook a cada push na `main`
- ✦ **ngrok** cria túnel HTTPS: URL pública → `localhost:8080`
- ✦ Jenkins recebe notificação e inicia pipeline
- ✦ Script `scripts/setup-ngrok.sh --webhook` automatiza a configuração



PIPELINE AS CODE — POR QUE IMPORTA PARA GCS

- ✦ O `Jenkinsfile` é um **IC versionado** no Git — a definição do pipeline evolui junto com o código
- ✦ Qualquer membro da equipe pode inspecionar e alterar o pipeline
- ✦ Mudanças no pipeline passam pelo mesmo fluxo de controle de mudanças (CCB)
- ✦ Rastreabilidade: cada build vinculado ao commit exato que o originou

TRIGGER

githubPush()

Webhook automático, sem polling

RASTREABILIDADE

IMAGE_TAG = BUILD_NUMBER

Artefato Docker vinculado ao build

AMBIENTE

venv isolado

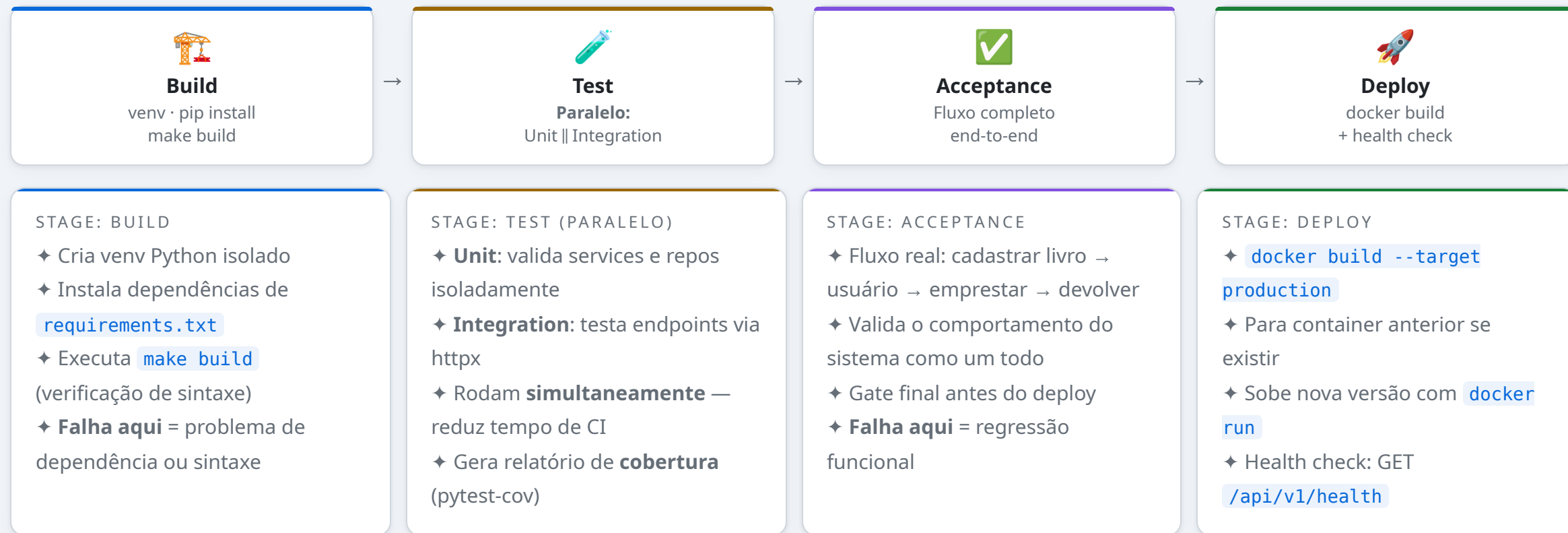
Cada build recria o ambiente Python

RESULTADO

SUCCESS / FAILURE

Gate automático de qualidade

Stages do Pipeline em Detalhe



Integração **GitHub** → **Jenkins** via ngrok

FLUXO COMPLETO — WEBHOOK

- 1. git push origin main**
Desenvolvedor empurra código para o repositório GitHub
- 2. GitHub dispara webhook**
Requisição HTTP POST para a URL pública do ngrok
- 3. ngrok encaminha para localhost**
Túnel seguro HTTPS → `localhost:8080` (Jenkins)
- 4. Jenkins inicia o pipeline**
Trigger `githubPush()` — clona repositório e executa stages
- 5. Resultado → histórico Jenkins**
`SUCCESS` = release · `FAILURE` = hotfix obrigatório



POR QUE NGROK?

- ✦ Sem servidor cloud — Jenkins roda **na máquina local**
- ✦ ngrok cria URL pública HTTPS que tunela para localhost
- ✦ GitHub consegue entregar webhooks sem acesso direto à rede local
- ✦ Solução **gratuita e funcional** para contexto acadêmico

Lições **Aprendidas**

01 Pipeline as Code transforma o Jenkinsfile em IC

Versionar o Jenkinsfile no Git nos forçou a tratar o pipeline com o mesmo rigor do código — mudanças passam pelo CCB e têm rastreabilidade completa.

02 ngrok exige disciplina operacional

O túnel deve estar ativo antes de qualquer demo. Aprendemos a automatizar com scripts e a manter o token válido — uma lição de confiabilidade de infraestrutura.

03 Testes paralelos exigem isolamento de estado

Rodar Unit || Integration no Jenkins economizou tempo, mas exigiu que cada teste crie e destrua seu próprio estado — boa prática que se pagou depois.

04 Gate automático (SUCCESS/FAILURE) exige commits pequenos

Aprovar o merge automaticamente pelo pipeline forçou commits menores e testes locais antes do push — a `main` ficou estável durante todo o projeto.

05 Gerência de Configuração é processo, não só ferramenta

Jenkins e Git são meios, não fins. O que realmente funcionou foi o fluxo: CCB, baselines formais e controle de mudanças com aprovação explícita.

Obrigado!

Dúvidas e perguntas são bem-vindas.

 github.com/leticiaafigueiredo/tp2-gerencia-conf

 PUC Minas · Gerência de Configuração e Evolução de Software · 2026